Experiment 2: Retrieval of Documents Using Inverted Files

Aim

To implement a program for retrieval of documents using an **Inverted Index (Inverted File)**.

Theory

What is an Inverted File?

An **Inverted File (Inverted Index)** is the most common data structure used by search engines to quickly locate documents containing a given word.

Instead of storing:

Document → Words

it stores

Word → Documents

This makes searching extremely fast.

Daily Life Example

Imagine a library.

Without Inverted File

To find books containing the word **"Python"**, the librarian opens every book one by one.

Time Required:

Book 1
Book 2
Book 3
...
Book 1000

Very slow.

With Inverted File

The librarian maintains an index.

Word	Books
Python	Book1, Book4, Book8
AI	Book2, Book4
Machine	Book3, Book5

Now searching takes only a few seconds.

Exactly how **Google** works.

What is an Inverted Index?

Suppose we have three documents.

Document 1

Python is easy to learn

Document 2

Python is used for AI

Document 3

AI makes life easy

After Tokenization

Doc1

Python
is
easy
to
learn

Doc2

Python
is
used
for
AI

Doc3

AI
makes
life
easy

Inverted Index

Word	Documents
python	D1, D2
easy	D1, D3
ai	D2, D3
used	D2
learn	D1
makes	D3
life	D3

Now,

Searching **AI**

Immediately returns

D2
D3

Algorithm

1. Read all documents.
2. Convert text into lowercase.
3. Tokenize into words.
4. Remove punctuation.
5. Create dictionary.
6. For every word
    - Check whether word already exists.
    - If not, create entry.
    - Add document ID.
7. Accept query from user.
8. Search query in inverted index.
9. Display matching documents.

Another Example

Input

easy

Output

Documents Found:
Doc1
Doc3

Time Complexity

Index Construction

O(N × M)
N = Number of documents
M = Average number of words per document

Searching

O(1) average (dictionary lookup)
O(k) to list matching documents, where *k* is the number of matching documents.

Advantages

Fast document retrieval.
Efficient searching in large collections.
Scalable to millions of documents.
Foundation of modern search engines like Google and Bing.
Supports Boolean, phrase, and ranked retrieval.

Limitations

Requires additional storage for the index.
The index must be updated whenever documents change.
Building the index can be time-consuming for very large datasets.

Applications

Search engines (Google, Bing)
Digital libraries
E-commerce product search

Email search
Document management systems
Hospital record search
Legal document retrieval

Viva Questions

1. What is an inverted file?
2. Why is an inverted index used in Information Retrieval?
3. What is the difference between a forward index and an inverted index?
4. How does an inverted index improve search speed?
5. What data structure is commonly used to implement an inverted index in Python?
6. What is tokenization?
7. Why are words converted to lowercase before indexing?
8. What is the time complexity of searching in an inverted index?
9. Name some real-world applications of inverted indexing.
10. How do search engines like Google use inverted indexes?